TECHIN 510 Lab 5 Report

Component A: Staff Interview

Interview Notes

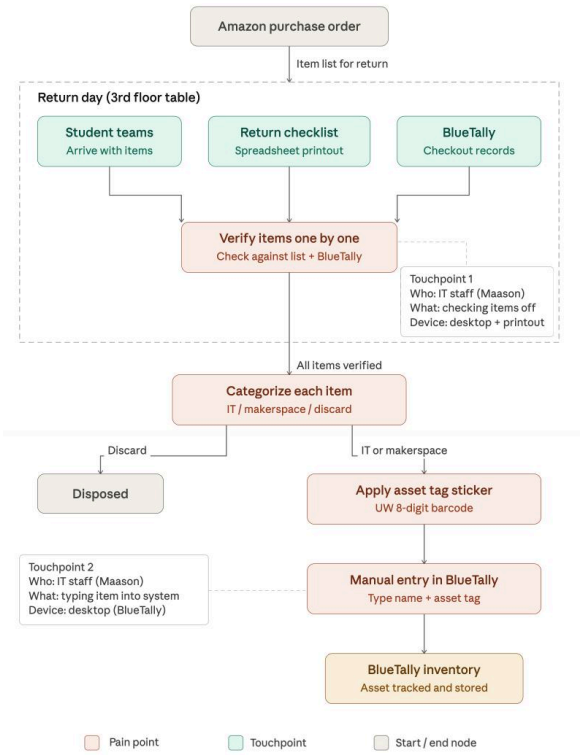
The current return process starts when Maason and Kevin set up a table on the third floor. Student teams arrive one at a time and Maason goes through a return checklist item by item, asking each team whether they have a given item to return. When a team confirms they have something, Maason marks it off the checklist. After all items from a team are verified, Maason and Kevin divide everything into three categories: items that belong to IT inventory, items that belong to the makerspace, and items that should be discarded because they were consumed as part of a prototype or are otherwise not worth keeping.

Any item going into inventory needs a barcode sticker applied. GIX uses pre-generated eight digit barcodes from a roll of stickers issued by the University of Washington. Once a sticker is on the item, Maason or Kevin enters it into BlueTally, the asset tracking software that GIX uses to manage equipment checkouts. BlueTally does support CSV import, but Maason rarely uses it for the return process because item names in the checklist come directly from Amazon product listings and are too long and messy. He wants short clean names like "Hollyland Mars M2 Wireless Mic" rather than a full Amazon description stuffed into a title field. Cleaning up those names one by one is a friction point that makes batch CSV import impractical right now.

On top of the returned purchased items, Maason also needs to check back in any items that were individually borrowed from BlueTally during the year. So on return day a team might have both purchased items to hand back and borrowed items to check in, and Maason handles both at the same table.

I think the core pain is not just that the process is slow, but that it requires too many manual transitions: cross reference the checklist, make a category judgment, find and apply a barcode, then re enter the item into BlueTally. Each of those is a separate step with no system connecting them.

System Map Sketch



Build Mandate

> "Based on the interview, I will build an equipment return tracker because Maason said going through every item one by one and re-entering it all into BlueTally takes too long, which means the app needs to speed up the check-in step and output data in a format ready for BlueTally import."

Component B: Lab

Tech Stack Justification

I chose Next.js with Supabase because the return workflow involves more then one staff member (Maason and Kevin or more) working at the same table and both need to read and write item records at the same time, which requires data to persist across sessions and users rather than staying local to one machine.

Supabase Schema Report

Column	Type	Default	Nullable
id	int8	auto	No
created_at	timestamptz	now()	No
item_name	text		No
team_name	text		No
category	text	it	No
status	text	pending	No
asset_tag	text		Yes
description	text		Yes

Responsive Design Summary

I tested the app at iPhone 14 Pro width using Chrome DevTools.

Element	Works at Phone Width?	What Breaks?
Page title	Yes	
Navigation	Yes	
Forms	Yes	
Tables	No	Item table overflows horizontally, columns get cut off
Buttons	Yes	
Text	Yes	

The most critical issue was the items table overflowing at phone width. I fixed it by wrapping the table in a div with `overflow-x-auto`, which allows horizontal scrolling instead of clipping the content.

Deployment URL

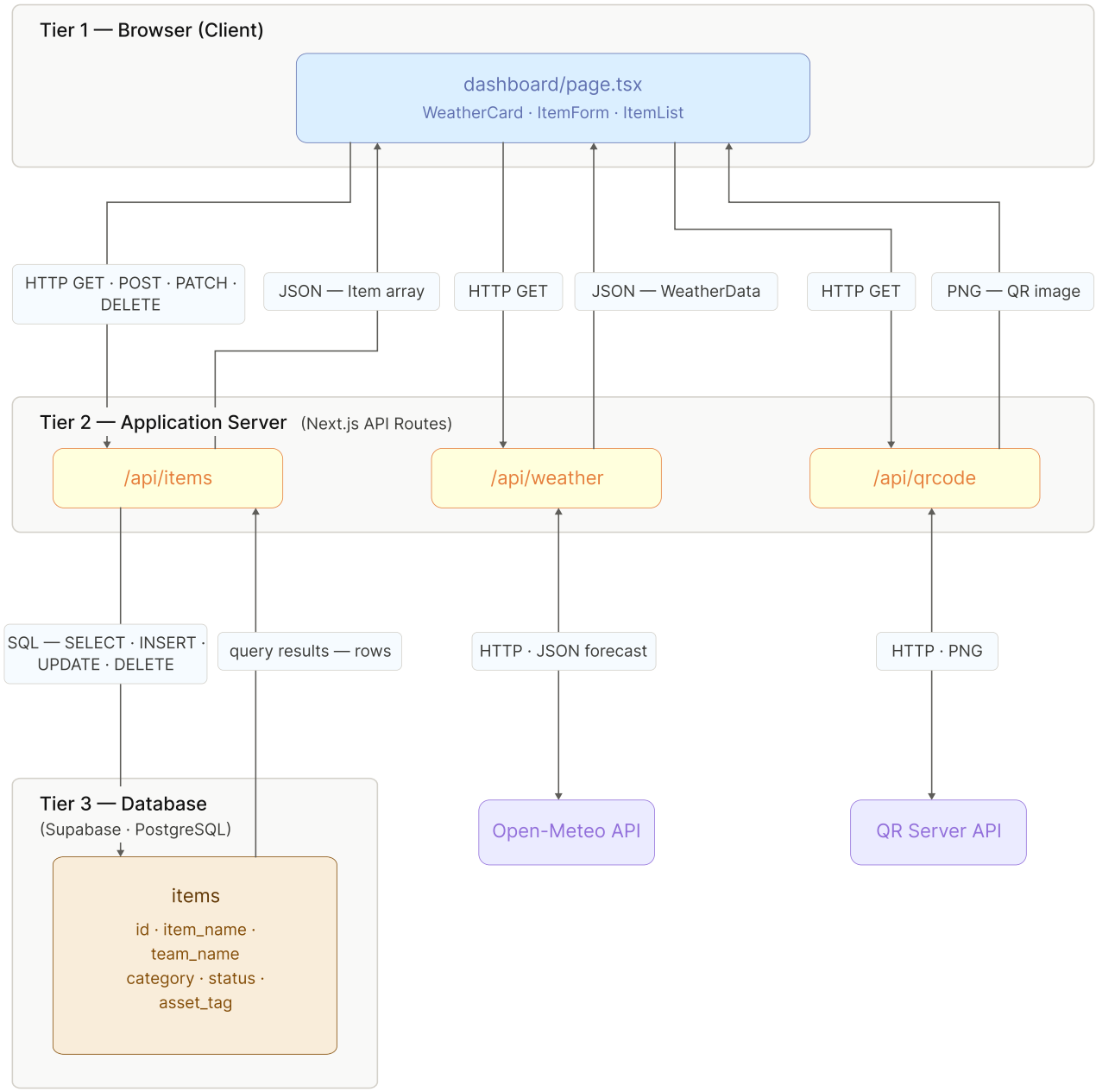
https://510-lab.vercel.app

Security Checklist

- ☑ No hardcoded secrets in source files — Supabase URL and anon key are stored in `.env.local` only
- ☑ `.env.local` is listed in `.gitignore` — confirmed not tracked by git
- ☑ Error handling on every API call and database operation — all fetch calls use try/catch with user-visible error messages; all Supabase queries check for `error` before using `data`

Component C: System Architecture and Design

Architecture Diagram



Design Decision Log

This Week's Decision Prompt

> Why did you choose this table structure (fields/relationships) for the workflow?

Field	Entry
Decision	I used a single "status" text column with three values (pending, returned, labeled) to track each item through the return workflow instead of using separate boolean columns or a separate status table.
Alternatives considered	Three separate boolean columns ("is_returned:, "is_labeled", "is_discarded"), or a separate status history table that logs every transition with a timestamp.
Why I chose this	The return workflow is strictly linear: an item always goes pending then returned then labeled, never skips a step or goes backward. A single ordered status column matches that reality and keeps queries simple.
Trade-off	I gave up a full audit trail. If Maason wants to know when exactly an item was marked returned, there is no timestamp for that transition. A status history table would capture that but adds complexity that is not needed for this use case.
When would I choose differently	If the workflow allowed items to go backward (e.g., an item marked returned could be sent back to pending after a dispute), a status history table would be the right choice.

Component D: Testing and Validation

Validation Results

#	Test Case	Input Description	Expected Outcome	Actual Outcome	Status Code	Pass/Fail
1	Valid input	Open-Meteo forecast request with latitude 47.65, longitude -122.30, daily temperature fields, 7-day forecast	200 OK, JSON with `daily` object containing `time`, `temperature_2m_max`, `temperature_2m_min` arrays	200 OK, received full 7-day forecast JSON with all expected fields	200	Pass
2	Invalid input	Open-Meteo request with latitude 999 (out of valid range)	400 Bad Request with error message about invalid latitude	400, `{"reason":"Latitude must be in range of -90 to 90°. Given: 999.0.","error":true}`	400	Pass
3	Missing/wrong auth	Supabase REST request with an incorrect anon key in the `apikey` and `Authorization` headers	401 Unauthorized with error message	401, `{"message":"Invalid API key","hint":"Double check your Supabase anon or service_role API key."}`	401	Pass

External API Assert Statements

The assert-style checks in `src/app/api/weather/route.ts` guard the app against Open-Meteo schema changes:

```
// Assert: response must contain "daily" with an array of dates
if (!data.daily || !Array.isArray(data.daily.time)) {
  return NextResponse.json(
    { error: "Unexpected weather data format from Open-Meteo" },
    { status: 502 }
  );
}

// Assert: required forecast arrays must be present
if (
  !Array.isArray(data.daily.temperature_2m_max) ||
  !Array.isArray(data.daily.temperature_2m_min)
) {
  return NextResponse.json(
    { error: "Missing temperature fields in weather response" },
    { status: 502 }
  );
}
```

The Supabase items route (`src/app/api/items/route.ts`) asserts that the query result is an array before returning it to the client:

```
if (!Array.isArray(data)) {
  return NextResponse.json(
    { error: "Unexpected data format from database" },
    { status: 500 }
  );
}
```

Error-Handling Note

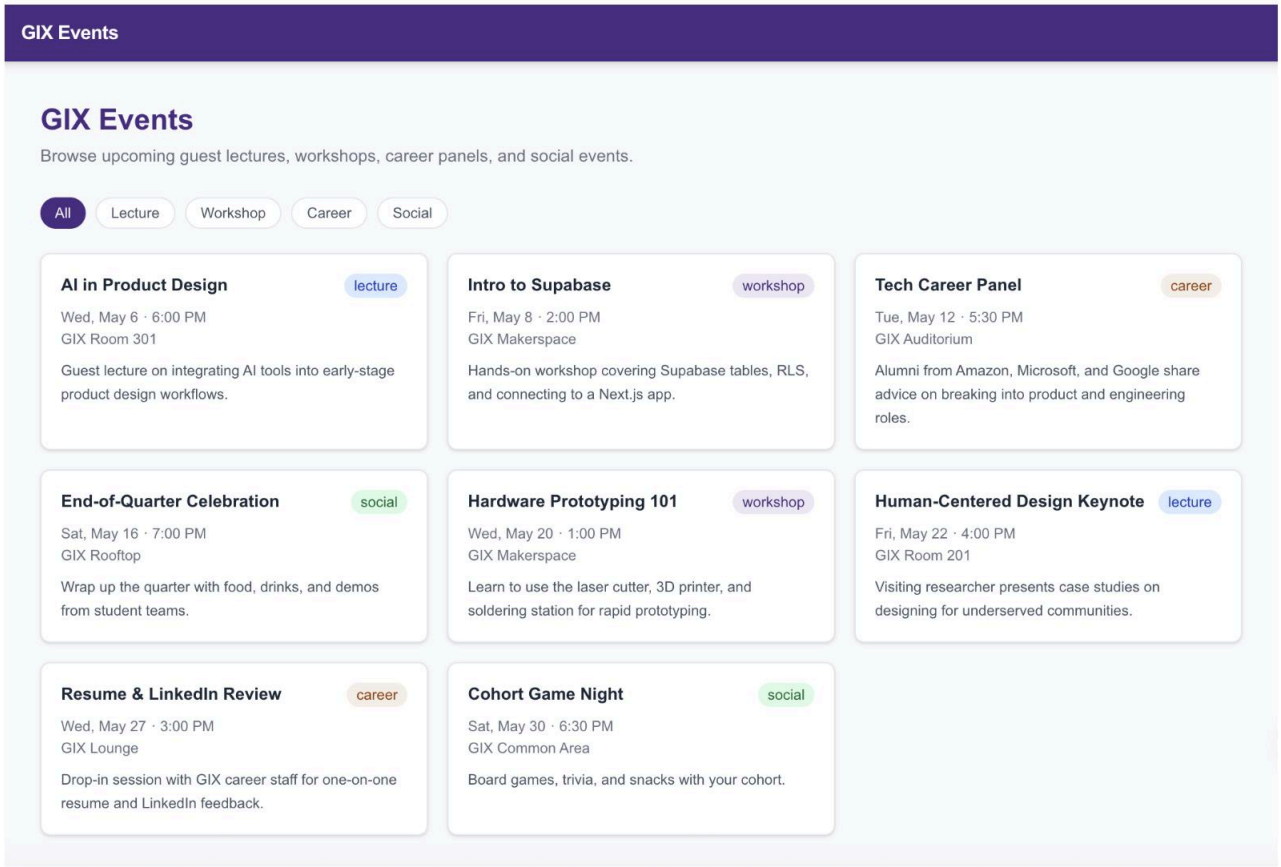
Scenario 1 — Valid input: Handled correctly. The happy path works end to end and the WeatherCard renders the 7-day forecast without error.

Scenario 2 — Invalid input (latitude 999): Open-Meteo returns 400 and the `/api/weather` route propagates the error as HTTP 502 with `{ "error": "Weather service unavailable" }`. The dashboard shows a user-visible error message rather than crashing. No gap found.

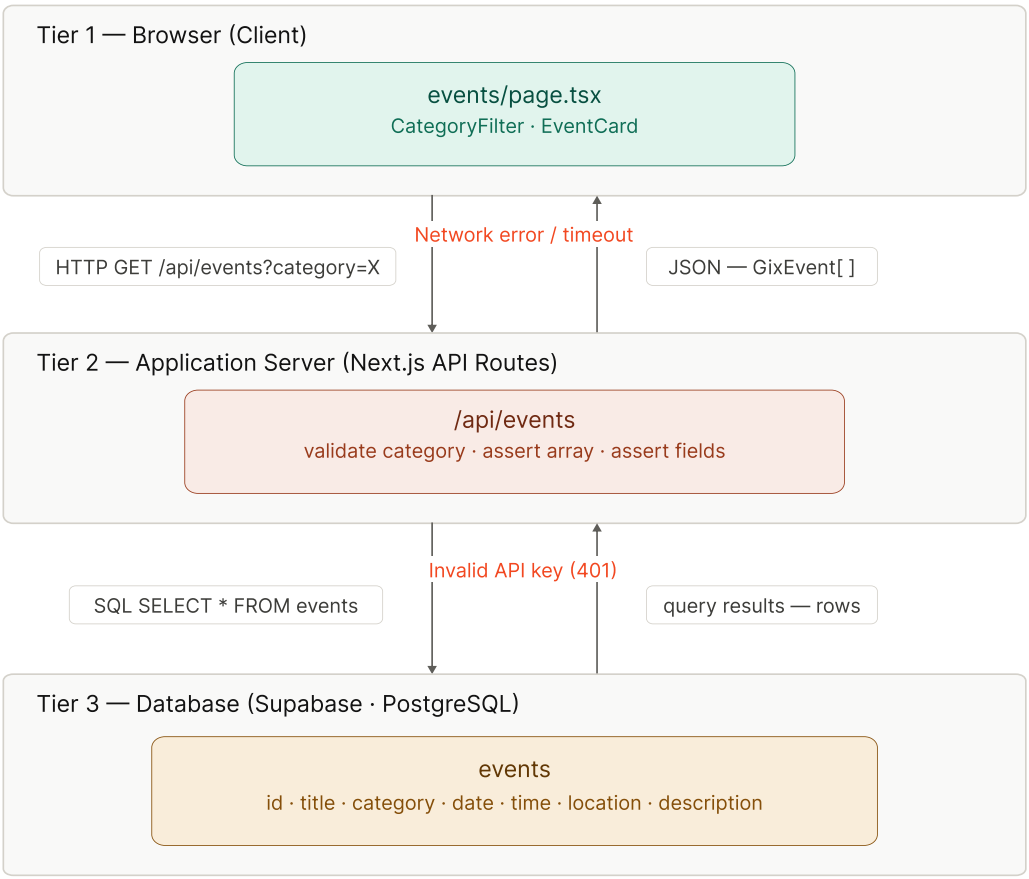
Scenario 3 — Missing/wrong auth: Supabase returns 401 when the anon key is invalid. The Supabase client library surfaces this as an `error` object on the query result. The `/api/items` route checks `if (error)` on every Supabase call and returns HTTP 500 with the error message. The dashboard shows the error inline. No gap found.

Component E: Applied Challenge

Live at: https://510-lab.vercel.app/events



System Architecture Map



Testing write-up

Assert Statements (events + Supabase pipeline)

Both asserts live in `src/app/api/events/route.ts`:

```
// Assert 1: Supabase must return an array
if (!Array.isArray(data)) {
  return NextResponse.json(
    { error: "Unexpected data format from database" },
    { status: 500 }
  );
}

// Assert 2: each event has required fields (id, title, category, date)
for (const event of data) {
  if (!event.id || !event.title || !event.category || !event.date) {
    return NextResponse.json(
      { error: "Event record missing required fields" },
      { status: 500 }
    );
  }
}
```

Error Scenario Tests

#	What I Did	Expected	Actual
1	Loaded `/events` with no matching events for a category (empty table state)	Page shows "No events found in the X category" empty state instead of crashing	Pass : empty state message renders correctly
2	Called `/api/events?category=invalid_value` with a category not in the allowed list	Route ignores the unrecognized value and returns all events (no crash, no 400)	Pass : `VALID_CATEGORIES` guard silently ignores unknown values and falls back to returning all events
3	Simulated a Supabase connection failure by temporarily replacing the anon key with a wrong value	`/api/events` returns 500 with `{ "error": "..." }`; events page shows red error banner	Pass : `if (error)` check on Supabase result surfaces the error as a user-visible message

Security Checklist

- [x] No hardcoded secrets in source files — Supabase URL and anon key are stored in `.env.local` only
- [x] `.env.local` is listed in `.gitignore` — confirmed not tracked by git
- [x] Error handling on every API call and database operation — all fetch calls use try/catch with user-visible error messages; all Supabase queries check for `error` before using `data`

AI Usage Log

Prompt 1:

> "Build a Next.js API route that connects to Supabase and supports GET, POST, PATCH, and DELETE for an items table with fields item_name, team_name, category, status, asset_tag, and description."

What it produced: A full `/api/items/route.ts` came back on the first try with all four HTTP methods, try/catch on every Supabase call, and required field validation on POST. The structure was solid and I did not have to rewrite anything major.

AI assumption: It assumed any status string was valid and did not enforce the order that items must go pending then returned then labeled. I never said the workflow was linear so it had no reason to know.

Failure mode: A client could PATCH an item straight to "labeled" without entering an asset tag, which would have broken the whole point of the labeling step.

What I would change: I would add one line to the prompt describing the allowed transitions so the API rejects out-of-order status updates from the start.

Prompt 2:

> "When an item's status is labeled, show a QR code image next to the asset tag in the items table."

What it produced: An `<img>` tag in ItemList.tsx pointing directly to the external QR Server API from the browser. It worked visually right away, which is why I did not catch it immediately.

AI assumption: It assumed any image request could go straight from the browser to an external service. I never mentioned the 3-tier architecture rule that all external API calls have to go through the server, so it had no reason to route it differently.

Failure mode: The browser was bypassing Tier 2 entirely and calling an external API directly, which violated the architecture requirement.

What I would change: I would include the constraint upfront: all external API calls must go through a Next.js server-side route, not the browser.

Prompt 3:

> "Build an item list where each row shows the current status and a button to advance it to the next state. Require the user to enter an asset tag before an item can be marked as labeled."

What it produced: `ItemList.tsx` with color-coded status badges, advance buttons that change label based on the current state, and an inline text input that appears inside the table row when an item is in the returned state.

AI assumption: It assumed the asset tag input should live directly inside the table row rather than in a modal or separate form. That seemed like the simplest approach and it never asked about screen size.

Failure mode: On a narrow screen the inline input pushes the asset tag column wider and makes the row hard to read, on top of a table that already needs horizontal scrolling on mobile.

What I would change: I would mention in the prompt that the table needs to stay readable on phone width, so the AI considers a modal or a drawer for the asset tag input rather than expanding an already crowded row.

Reflection

Transition: What surprised you about the difference between building in Streamlit versus Next.js? What felt easier? What felt harder? How does reading AI-generated Python compare to reading AI-generated TypeScript?

The biggest surprise going from Streamlit to Next.js was how much mental overhead came from having to decide where every piece of code lives. In Streamlit I just wrote Python and it worked, but in Next.js I kept asking myself whether something should be a server component, a client component, or an API route, and getting it wrong caused actual bugs rather than just style issues. Reading AI-generated TypeScript also felt harder than reading AI-generated Python because

the types and imports make every file look longer than it is, though once I got used to the structure I could actually follow the data flow more easily.

System map: What did the system map reveal about the interviewee's pain that you would not have understood from the interview questions alone?

Drawing the system map helped me see something the interview did not make obvious on its own: Maason has to context-switch between four different tools in a fixed order every single return day, and none of them talk to each other, so the pain is not just one slow step but the whole chain adding up every time. The JTBD statement from Week 4 captured what he wanted to achieve, but the system map showed exactly where things break down in a way I could actually design around.

Tech stack choice: When is Streamlit the right tool, and when is Next.js with Supabase the right tool?

I would use Streamlit when I need to show a quick data visualization to one person, like a weekly usage chart, and Next.js with Supabase when multiple people need to read and write the same data at the same time, like this return tracker where Maason and Kevin both need to update items at the same table.